GROUP 21 · CODEQUEST 2026

NestBridge

Complete User & Technical Guide

How to use the app · How it was built · Tech stack
Run commands · Architecture · Judge Q&A

Culturally intelligent pairing for students, hosts, guides & tourists
Prototype: mellow-blini-a42359.netlify.app · July 2026

Table of Contents

1. What NestBridge Is

NestBridge is a culturally intelligent pairing platform that connects international students with host families, and tourists with local guides and lodging — in one shared account model.

Built by Group 21 for CodeQuest 2026. Finding a stay or guide should feel like cultural fit, not just a hotel search. Matching considers language, diet, lifestyle, budget, proximity, cultural affinity, and trust.

Who it is for

Primary intent	Home focus	Also can
STUDENT	Book host families	Book guides; browse lodging
TOURIST	Guides + lodging explore	Book homestays
HOST	Incoming stay requests	Travel: book guides/stays
GUIDE	Incoming session requests	Travel: book stays/guides
Staff / Admin	Moderation tools	Suspend, KYC flags, audits

One person = one account. Primary intent only personalizes the home
(hard exclusion). Other services are additive via Account Setup tracks (

2. End-User Guide (How to Use the App

2.1 First launch & account

- Open NestBridge (Expo Go or built APK).
- Welcome !’ Create account (name, email, password only — no role
- Verify email if required (local/dev often auto-skips when SendGrid is
- Choose primary intent: Student / Tourist / Host / Guide.
- Complete short onboarding (destination, quiz, profile). Soft skips ex

2.2 Student flow (homestay)

- Home !’ Search hosts (destination/dates may be pre-filled from onb
- Match Results show: compatibility %, trust badge, and "e2 plain-En
- Open Host Profile !’ Message (optional) !’ Request to book.
- Booking screen shows nightly rate + platform fee + total + cancellat
- Host accepts !’ Pay (Paystack live, or mock confirm in demo) !’ Conf

2.3 Tourist flow

- Explore home !’ find guides or lodging directory.
- Book guide sessions: PENDING !’ ACCEPT !’ PAY !’ CONFIRMED.
- External lodging partners: call / email / website — no NestBridge pa

2.4 Host flow

- Complete host listing (property, amenities, photos, availability, optio
- Dashboard shows incoming requests — accept or decline.
- Max 2 overlapping guest stays at once.

2.5 Guide flow

- Set services, bio, weekly schedule, optional social verify + KYC.
- Review incoming session requests; max 2 overlapping time blocks.

2.6 Shared tabs (after login)

Tab	Purpose
Home	Intent-specific dashboard
Search	Unified search: homestays, guides, lodging
Bookings	Outgoing + provider review queue

Tab	Purpose
Messages	Firebase (or API) chat
Profile	Account setup, settings, sign out

2.7 SOS & welfare (non-negotiable)

- Floating SOS button is always visible on authenticated screens — never hide it.
- SOS screen: emergency contacts, call actions, logs event to backend with optional location.
- Welfare check-ins can be logged against a booking.

2.8 Ghana content library

Phrases, transport tips, tourist sites, checklist, videos, map landmarks — loaded from backend content demo merge if sparse for judges).

3. Demo Accounts for Judges

Demo mode is ON when EXPO_PUBLIC_ENABLE_DEMO_FALLBACK=true (development/preview). Login/Register show quick-login tiles.

Role	Email	Password
Student	akosua.demo@nestbridge.app	password
Tourist	zara.tourist@nestbridge.app	password
Host	abena.host@nestbridge.app	password
Guide	kofi.guide@nestbridge.app	password

All @nestbridge.app accounts are email-pre-verifie merges only to fill sparse responses for demos.

Judge 3-minute path

- Start Postgres+Redis !’ backend !’ Expo.
- Welcome !’ Student quick-login !’ Search !’ Matcl
- Logout !’ Host quick-login !’ Accept request.
- Tap SOS floating button !’ show emergency cont

4. How the App Was Created

Two-phase build strategy

- Phase 1 — Presentational screens: each screen is a pure TypeScript component; data arrives via typed props. Teammates built screens in parallel against the Netlify prototype (~29 screens). Mock data lived outside screen files.
- Phase 2 — Integration (Bless wiring pass): navigation, Axios API client, matching endpoint, Firebase chat, booking/payment, SOS, real backend. No fake API in production navigation paths.

Assignment constraints we followed

- Expo / React Native + TypeScript only for new source.
- StyleSheet.create only — no NativeBase, Paper, Tamagui, NativeWind.
- All colors/spacing/radii from src/constants/theme.ts.
- SOS always visible; match % + trust + "e2 reasons; booking price breakdown.
- KYC/photo soft skip (“Verify later” / “Skip for now”).

Prototype !’ product

UI structure was matched to the prototype (mellow-blini-a42359.netlify.app). Brand tokens stayed in theme.ts — we never sampled hex codes from screenshots. The codebase grew beyond 29 screens (auth extras, admin/staff, lodging, content, settings) while keeping the prototype flows intact.

Team model

- Screens could be built by anyone first; ownership lists are “who builds first,” not scope walls.
- Matching algorithm, backend endpoints, Firebase wiring, and final navigation integration were Bless’s responsibility for the demo-critical path.

5. Prescribed Tech Stack & Why Each Piece

Frontend (mobile)

Technology	Version / note	Why
Expo	~54	Fast RN builds, Expo Go demos
React Native	0.81	iOS + Android one codebase
React	19.1	UI layer
TypeScript	5.x	Typed props & safer APIs
React Navigation	native-stack	Auth + app stacks
Axios	api.ts	Single HTTP client + JWT
Expo SecureStore	tokens	Safer than AsyncStorage
Firebase JS SDK	RTDB chat	Realtime messaging only
AsyncStorage	cache	Offline content cache
Poppins fonts	Expo Google Fonts	Brand typography

Backend (API)

Technology	Version / note	Why
Java	17	Assigned LTS runtime
Spring Boot	3.3.5	REST, Security, JPA
Spring Security + JWT	jjwt 0.12.6	Stateless auth
Spring Data JPA	Hibernate	ORM; ddl-auto=validate
Flyway	migrations	Schema versioning
PostgreSQL	15	Primary relational store
Redis	7	Token blacklist + match cache

Technology	Version / note	Why
Firebase Admin	optional	Provision chat nodes
Paystack / Smile / S3	feature flags	Pay, KYC, photos
Maven Wrapper	mvnw	Reproducible builds

Local infra: Docker Compose in backend/docker-compose.yml (Pos
Health: GET /actuator/health.

6. Architecture — How Front, Back & D

Request path (say this in one breath)

The Expo app calls Axios (api.ts) !' Spring Boot controllers on :8080
caches match results and blacklists refresh tokens. Firebase Real
still store messages for audit.

Layer diagram

```
[ Expo / React Native screens ]
        |  props + AuthContext / SecureStore JWT
        v
[ src/services/api.ts  (Axios + Bearer token) ]
        |  HTTP JSON  ApiResponse<T>
        v
[ Spring Boot Controllers  /api/... ]
        |
        +--> Auth (JWT filter) / Matching / Booking / Welfare / Admin
        +--> PostgreSQL (Flyway V1..V32)
        +--> Redis (match cache, refresh blacklist)
        +--> Firebase RTDB (chat)  [optional]
        +--> Paystack / Smile / S3 / SendGrid  [flags]
```

Unified account model

Register creates a user with no role. IntentSelect sets primary_intent. Seeker profile enables booking. Host/guide provider setups enable accepting requests. Browse is always allowed; book/pay/accept gate on COMPLETE setup tracks.

7. Exact Build & Run Commands

Prerequisites

- Node.js 18+ and npm
- Java 17 JDK
- Docker Desktop (for Postgres + Redis)
- Expo Go on a phone, or Android emulator / iOS simulator

7.1 Start database + Redis

```
cd backend
docker compose up -d

# Postgres :5432  Redis :6379
# DB nestbridge / user postgres / password postgres
```

7.2 Start Spring Boot backend

```
cd backend

# Windows PowerShell:
.\mvnw.cmd spring-boot:run

# macOS / Linux:
./mvnw spring-boot:run

# Health check !’ http://localhost:8080/actuator/health
```

If Flyway checksum errors appear after editing migrations:

```
./mvnw flyway:repair
./mvnw spring-boot:run
```

7.3 Start Expo frontend

```
cd frontend
npm install
npm start
# same as: npx expo start

# scan QR with Expo Go (same Wi-Fi)
# press a !’ Android   press i !’ iOS
# npm run android / ios / web
```

API base URL defaults toward localhost:8080. On a physical device, env.ts uses the Metro LAN IP so the phone can reach your PC. Override with EXPO_PUBLIC_API_BASE_URL if needed.

7.4 Useful Expo commands

```
cd frontend
npx expo install
npx expo doctor
eas build --profile production --platform android
```

7.5 Full local stack order (memorize)

```
1) cd backend && docker compose up -d
2) cd backend && .\mvnw.cmd spring-boot:run
3) cd frontend && npm start
4) Expo Go !’ demo quick-login
```

8. Matching Algorithm (Deep Dive)

Source: backend/.../matching/MatchingAlgorithm.java. Endpoint: POST /api/matches/find. Cached in Redis as match:{userId}:{searchHash} for ~15 minutes.

Host matching

- Hard filters first: active listing, city, max budget, hard dietary (halal/kosher/allergy).
- Weighted score (0–100): language 20% + diet 20% + lifestyle 15% + budget 15% + proximity 15% + cultural 10% + trust 5%.

Guide matching

- language 30% + budget 20% + proximity 20% + trust 15% + service 15%.

UI must show

- Compatibility percentage + trust badge + at least two plain-English matchReasons.

9. Auth, Booking, Payments, SOS, Chat

9.1 Authentication

- POST /api/auth/register !’ create user (BCrypt password).
- Email verify token (SendGrid) or auto-skip when EMAIL_VERIFICATION_ENABLED=false.
- POST /api/auth/login !’ access JWT (~15 min) + refresh (~30 days).
- Access token in Authorization: Bearer … from SecureStore.
- POST /api/auth/refresh-token; logout blacklists refresh in Redis.
- Staff emails on ADMIN_EMAIL_ALLOWLIST get is_staff for admin screens.

9.2 Booking lifecycle

```
Discover !’ Profile !’ (optional) POST /api/conversations
POST /api/bookings              !’ PENDING_HOST
Provider GET /api/bookings/incoming !’ accept | decline
Accept !’ ACCEPTED !’ guest Pay now
Paystack ON:  /payment/initialize !’ checkout !’ /payment/verify
Paystack OFF: PUT /api/bookings/{id}/confirm !’ CONFIRMED
```

9.3 SOS

- UI: shared SOSScreen + floating button in AppNavigator wrapper.
- Contacts: GET /api/content/emergency-contacts
- Log: POST /api/welfare/sos (lat/lng, contacted flags)
- Optional email alert via SUPPORT_ALERT_EMAIL + SendGrid.

9.4 Chat

- Firebase is for messaging only — not auth, not bookings, not profiles.
- Backend can provision conversation nodes; client listens on /conversations/{id}/messages.
- Fallback: Postgres-backed GET/POST /api/conversations/:id/messages if Firebase is off.

10. Database & Migrations

PostgreSQL is the system of record. Hibernate validates against the schema; Flyway owns changes (V1 through V32+). Never rely on ddl-auto=update in production.

Core tables (conceptual)

- users — identity, primary_intent, staff flags, email verified
- seeker_profiles / provider_setup — travel vs listing readiness
- host_profiles / guide_profiles — provider listings
- matches / bookings — pairing + lifecycle status
- welfare_check_ins / sos_events — safety
- Later: chat audit, content, events, payments, KYC, reviews, notifications, staff audit

Local connection

```
jdbc:postgresql://localhost:5432/nestbridge
username: postgres
password: postgres
```

11. Key API Endpoints

Base URL local: http://localhost:8080. Responses wrap as ApiResponse<T>.

Area	Examples
Auth	/api/auth/register, /login, /refresh-token, /logout
Users	/api/users/me/profile, device-tokens
Matches	POST /api/matches/find

Area	Examples
Bookings	POST /api/bookings, /incoming, /accept, /confirm
Hosts/Guides	/api/hosts|guides/{id}, /profile
Chat	/api/conversations, /{id}/messages
Welfare	/api/welfare/sos, /checkins/{bookingId}
Content	/api/content/phrases|transport|sites|...
Admin	/api/admin/overview, users/search, suspend
Health	/actuator/health

12. Project Folder Map

```
nestbridge/
% % %  .cursorrules              # assignment + design rules
% % %  .env.example              # full env catalog
% % %  docs/                     # AGENTS, Backend, Demo, Production
% % %  backend/
%    % % %  docker-compose.yml    # Postgres 15 + Redis 7
%    % % %  pom.xml               # Spring Boot 3.3.5 / Java 17
%    % % %  mvnw / mvnw.cmd
%    % % %  src/main/
%       % % %  java/com/nestbridge/{auth,matching,booking,...}
%       % % %  resources/db/migration/   # Flyway
% % %  frontend/
    % % %  package.json          # npm start !’ expo start
    % % %  app.config.ts
    % % %  src/
       % % %  screens/{auth,onboarding,student,host,guide,tourist,shared}
       % % %  services/{api.ts,firebase.ts}
       % % %  navigation/, context/, constants/theme.ts
       % % %  config/{env.ts,demoMode.ts}
```

13. Design System & UX Rules

- Tokens only from frontend/src/constants/theme.ts (colors, fonts, spacing, radii, gradients).
- Header/splash !’ gradients.header; CTAs !’ teal; backgrounds !’ background / warmCream.
- Danger/SOS !’ colors.danger; success !’ colors.success; warnings !’ colors.warning.
- Minimum touch target 44×44pt.
- Loading: ActivityIndicator — never blank screens; unreachable API !’ clear error (except explicit demo merge).

14. Judge Q&A — Prove You Built This

Use these answers in your own words. They map to real files in this repo.

Q: What problem does NestBridge solve?
A: International students and tourists struggle to find culturally compatible stays and guides. We match on language, diet, lifestyle, budget, proximity, culture, and trust — not just price and photos.

Q: What is your tech stack?
A: Mobile: Expo 54 + React Native + TypeScript + React Navigation + Axios + SecureStore + Firebase RTDB for chat. Backend: Java 17, Spring Boot 3.3.5, Spring Security JWT, JPA, Flyway, PostgreSQL 15, Redis 7. Payments Paystack; KYC Smile; photos S3 — all feature-flagged.

Q: Why Redis if you already have Postgres?
A: Postgres holds durable domain data. Redis is for short-lived concerns: blacklisting refresh tokens on logout and caching expensive match results for ~15 minutes.

Q: Why Firebase and also Spring?
A: Spring is the source of truth for users, bookings, matching, welfare. Firebase Realtime Database is only for low-latency chat. If Firebase is off, chat falls back to Postgres message endpoints.

Q: Walk me through a booking.
A: Seeker creates POST /api/bookings (PENDING_HOST). Provider accepts !’ ACCEPTED. Guest pays via Paystack initialize/verify or mock confirm. Status becomes CONFIRMED. Hosts/guides limited to 2 overlapping bookings.

Q: How does matching work?
A: Hard filters (city, budget, dietary) then weighted soft scores. Hosts: language/diet/lifestyle/budget/proximity/cultural/ trust. Guides: language/budget/proximity/trust/service. API returns percent, breakdown, and plain-English reasons.

Q: Where is the JWT stored and why?
A: Expo SecureStore on device — encrypted keychain/keystore style storage. Not AsyncStorage. Axios attaches Bearer token; refresh rotates; logout blacklists refresh in Redis.

Q: How do I run your demo right now?
A: docker compose up -d in backend, mvnw spring-boot:run, then npm start in frontend. Use Welcome quick-login tiles with password “password” for the four @nestbridge.app demo users.

Q: What did YOU personally build / own?
A: Be honest to your real work. Strong talking points if they apply: MatchingAlgorithm.java, api.ts wiring, AppNavigator + SOS wrapper, booking/payment flow, Flyway seeds, demo mode flag, staff admin API. Point at file paths.

Q: Why no UI component library?
A: Assignment rule: raw StyleSheet + theme.ts tokens for consistency and to prove we control the design system.

Q: How is schema evolved?
A: Flyway migrations under backend/src/main/resources/db/migration. Hibernate set to validate — the DB schema must already match entities.

Q: What happens if the backend is down?
A: Screens show a clear error / failed login — we do not silently invent bookings. Demo mode may merge content for sparse successful responses, but auth and writes still need the API.

Q: Security highlights?
A: BCrypt passwords, short-lived access JWT, refresh rotation + Redis blacklist, staff allowlist, Spring Security filter chain, optional email verification, webhook endpoints for Paystack/Smile.

Q: What is still env-dependent for production?
A: SendGrid email verify, Paystack live keys + webhook, Smile KYC, S3 photos, Firebase credentials, JWT_SECRET, hosted Postgres/Redis, EXPO_PUBLIC_ENABLE_DEMO_FALLBACK=false for production builds.

15. Troubleshooting

Symptom	Fix
Phone cannot hit API	Use LAN IP in EXPO_PUBLIC_API_BASE_URL; same Wi-Fi
Docker not running	Start Docker Desktop; docker compose up -d
Port 8080 in use	Stop other Java apps or change server.port
Flyway checksum fail	mvnw flyway:repair then restart
Demo login fails	Backend must be up; seeds applied; demo flag true
Blank Expo	npm install; npx expo start -c
JWT 401 loops	Re-login; check Redis is up

Appendix A — Memorize These Commands

```
cd backend
docker compose up -d
.\mvnw.cmd spring-boot:run

cd frontend
npm install
npm start                # expo start

# Health
curl http://localhost:8080/actuator/health
```

Appendix B — Primary Docs in Repo

- docs/AGENTS.md — product, screens, UX, navigation
- docs/Backend.MD — schema, matching, API, JWT
- docs/DEMO_AND_PRODUCTION.md — judge demo mode
- docs/PRODUCTION_SETUP.md — production env checklist
- docs/DEPLOYMENT.md — deploy notes
- backend/README.md — local backend run
- .cursorrules — CodeQuest constraints
- .env.example — environment variable catalog

Appendix C — Elevator Pitch

NestBridge is Group 21’s CodeQuest 2026 culturally intelligent pairing app: Expo/React Native on the phone, Spring Boot on the server, PostgreSQL for truth, Redis for cache and token safety, and Firebase only for live chat. Students and tourists discover hosts and guides with explainable match scores; hosts and guides manage incoming requests; everyone keeps SOS one tap away. We built screens to the prototype, then wired real auth, matching, bookings, and welfare so judges can run the full stack locally with Docker, Maven, and Expo.

Generated for NestBridge Group 21 — use this guide to demo, defend, and operate the stack.